

CS 450: OPERATING SYSTEMS

Programming Assignment 1: Process Creation

Due date: 04th September 2026

Understanding parent–child relationships and process hierarchies

WHAT THIS ASSIGNMENT IS REALLY ABOUT

`fork()` is four letters long and is the single most confusing call in this course, because it is the only function you will ever write that **returns twice**. Everything here follows from taking that sentence literally. By the end you will have built process trees of arbitrary shape, watched a child outlive its parent, and seen a dead process sit in the process table waiting to be collected.

1 Background

1.1 The call that returns twice

`fork()` creates a near-identical copy of the calling process. Both processes resume at the same instruction (the one after the `fork()`) — with the same variables, the same open file descriptors and the same buffered output. The **only** immediate difference is the value `fork` returned:

<code>fork()</code> returned	You are	What you should do
0	the child	do the child's work, then <code>_exit(code)</code>
<code>> 0</code>	the parent	the value is the child's PID; remember it and <code>waitpid()</code> for it
<code>< 0</code>	still the parent	the fork failed — check <code>errno</code> and handle it

1.3 Ending a child correctly

How the child ends	Effect	Use it?
<code>_exit(n)</code>	terminates immediately; does not flush <code>stdio</code>	yes — this is the rule for children
<code>exit(n)</code>	runs <code>atexit</code> handlers and flushes <code>stdio</code> — including buffered text inherited from the parent	no, unless you know the buffer is clean
<code>return n</code> from <code>main</code>	identical to <code>exit(n)</code>	no, for the same reason

2 Part 1 — The Program (70 points)

Write one executable, `procs`, with subcommands. Each builds a process tree and prints **records** describing it.

2.1 Command line

```
./procs SUBCOMMAND [ARGS]

fork1          one child; show fork()'s two return values
fanout N        N children of a SINGLE parent           (0 <= N <= 200)
chain N         a chain N links deep                     (0 <= N <= 200)
tree K D        a complete K-ary tree of depth D         (1<=K<=8, 0<=D<=6)
status          exit codes, and a child killed by a signal
execdemo        fork + exec; show the PID survives
orphan          a child outlives its parent
zombie          an unreaped child sits in state Z
helper LABEL    internal: the program execdemo runs
```

Any other subcommand, a missing argument, or an argument outside its range must print a message to `stderr`, print **no** records, and exit **non-zero**.

2.2 The record format — the output contract

Exactly three record types, one per line, no other output on stdout:

```
NODE <label> pid=<pid> ppid=<ppid>
    printed by a process ABOUT ITSELF, once, as soon as it knows its role

REAP <label> pid=<pid> status=exited:<code>
REAP <label> pid=<pid> status=signaled:<signo>
    printed by a PARENT after wait()/waitpid() returns for that child

INFO <key>=<value>
    one requested fact
```

2.3 Labels and required INFO keys, per subcommand

Subcommand	Labels	Required INFO
fork1	root, c0	parent_sees_fork_return_is_child_pid, child_sees_fork_return, child_getppid_equals_root
fanout N	root, c0 ... c(N-1)	children_created · child * i must_exit(i % 100)
chain N	n0 ... n<N>	chain_depth
tree K D	r, then <parent>.<index> — e.g. r.0, r.1.0	expected_nodes
status	root, e0, e1, e2, sig	exit_status_is_8_bits · e0/e1/e2 exit 0/1/42; sig is killed with SIGTERM
execdemo	root, c0 (before exec), execed (after)	parent_recorded_child_pid, child_pid_before_exec, pid_after_exec
orphan	root, mid, orphan	orphan_ppid_before, orphan_ppid_after
zombie	root, z	state_before_wait, state_after_wait

2.4 Requirements that apply everywhere

- Every parent must reap every child it creates, before it exits.
- Every child must end with ``_exit()'``, never by falling out of a loop and never with `exit()`.
- `fork()` can fail. Check for a negative return and report it rather than treating the failure as if you were the child.
- `tree` builds a **complete** K-ary tree: every internal node has exactly K children, and every leaf is at depth D.

3 Worked Example

Run `./procs fork1`. Your PIDs will differ; the *relationships* will not.

```
NODE root pid=11 ppid=10
INFO parent_sees_fork_return_is_child_pid=12
NODE c0 pid=12 ppid=11
INFO child_sees_fork_return=0
INFO child_getppid_equals_root=1
REAP c0 pid=12 status=exited:0
```

Read it as a tree: `root` forked once, so `c0`'s `ppid` equals `root`'s `pid`. The parent saw `fork` return `c0`'s PID; the child saw `0`. The parent then waited and recorded that `c0` exited with code 0.

A chain three links deep

```
NODE n0 pid=14 ppid=13
NODE n1 pid=15 ppid=14
NODE n2 pid=16 ppid=15
NODE n3 pid=17 ppid=16
REAP n3 pid=17 status=exited:0
REAP n2 pid=16 status=exited:0
REAP n1 pid=15 status=exited:0
INFO chain_depth=3
```

Each `ppid` is the previous node's `pid`, so the tree is a straight line. Note the `REAP` records unwind in reverse: `n2` cannot finish waiting for `n3` until `n3` has exited.

A complete binary tree of depth 2

```
NODE r pid=19 ppid=18
NODE r.0 pid=20 ppid=19
NODE r.1 pid=21 ppid=19
NODE r.0.0 pid=22 ppid=20
NODE r.0.1 pid=23 ppid=20
NODE r.1.0 pid=24 ppid=21
NODE r.1.1 pid=25 ppid=21
REAP r.0.0 pid=22 status=exited:0
REAP r.0.1 pid=23 status=exited:0
REAP r.1.0 pid=24 status=exited:0
REAP r.1.1 pid=25 status=exited:0
REAP r.0 pid=20 status=exited:0
REAP r.1 pid=21 status=exited:0
INFO expected_nodes=7
```

Seven nodes: $1 + 2 + 4$. The `NODE` lines appear in a different order every run — the shape does not change.

4 The Three Demonstrations

Three subcommands exist to make an invisible kernel behaviour visible.

4.1 ``execdemo`` — `exec` replaces the program, not the process

```
NODE root pid=27 ppid=26
INFO parent_recorded_child_pid=28
NODE c0 pid=28 ppid=27
INFO child_pid_before_exec=28
NODE execed pid=28 ppid=27
INFO pid_after_exec=28
REAP c0 pid=28 status=exited:0
```

The parent recorded the child's PID, the child printed its own PID before calling `exec`, and the *newly loaded program* printed its PID after. **All three are the same number.** `exec` overwrote the code, data, heap and stack of that process, and kept everything the kernel tracks about it: its PID, its parent, and its open file descriptors.

4.2 `orphan` — a child outliving its parent

```
NODE root pid=30 ppid=29
NODE mid pid=31 ppid=30
REAP mid pid=31 status=exited:0
NODE orphan pid=32 ppid=31
INFO orphan_ppid_before=31
INFO orphan_ppid_after=1
INFO orphan_reaped_by=1
```

`orphan`'s parent was `mid`. When `mid` exited, the kernel re-parented `orphan` to **PID 1**, and its `getppid()` changed underneath it while it was running. Nothing else about it changed. This is not an error — it is how every daemon on the machine got started.

4.3 `zombie` — a dead process that has not been collected

```
NODE root pid=34 ppid=33
NODE z pid=35 ppid=34
INFO state_before_wait=Z
REAP z pid=35 status=exited:7
INFO state_after_wait=?
```

The child called `_exit(7)` and stopped running, but the kernel cannot discard it: somebody may still ask how it ended. Until the parent calls `wait`, it remains in the process table in state ``Z``, holding nothing but its exit status. After the `wait`, `/proc/<pid>/stat` no longer exists — the state reads `?` because the process is gone.

5 Part 2 — The Written Report (30 points)

Submit [report.pdf](#), at most 2 pages. Every claim must be supported by output your own programs produced — paste the lines/screenshots or include a table you generated. Answers with no evidence score zero, however fluent.

Q1 `_exit` versus `exit` (10 points)

In your process-creation program, explain the difference between `_exit()` and `exit()` when a child process terminates. Why is `_exit()` commonly preferred in the child process after a failed `exec()` call? In your explanation, discuss what happens to `stdio` buffers when `fork()` is followed by `exit()`, and how using `_exit()` can prevent the child from flushing inherited buffers a second time.

Q2 Ordering is not a contract/guarantee (10 points)

Run `./procs fanout 6` at least 10 times, capturing the order of the `NODE` lines each time. Report how many **distinct orderings** you saw and how many distinct **tree shapes**. Explain why the first number (**distinct orderings**) is greater than one and the second (distinct **tree shapes**) is not. Then name two mechanisms that *would* impose a deterministic order.

Q3 Orphans, zombies and cleanup (10 points)

Using your own `orphan` and `zombie` output, fill in a table contrasting the two on: *is it running?*, *what is its ppid?*, *what resource does it hold?*, *who finally cleans it up?*, and *what happens if nobody does?*. Then list three distinct ways a long-running server can avoid accumulating zombies.

6 Deliverables and Submission

File	Contents
<code>procs.c</code>	your implementation — the only source file you must write
<code>procs.h</code> , <code>main.c</code> , <code>Makefile</code>	unmodified from the starter, unless you documented a change
<code>report.pdf</code>	the written report, at most 2 pages
<code>README.md</code>	how to build; anything unfinished; all help received
video recording	Demo each of the subcommand in <code>./procs</code>

Please submit your work in two locations:

- **GitHub:** Submit your work to the GitHub repository using the link provided through the invitation.
- **Canvas:** Place all your assignment files in one folder, compress (zip) the folder, and upload the resulting ZIP file to Canvas under the designated assignment submission link.

Please make sure that your GitHub submission and Canvas ZIP file contain the required assignment files.

7 Grading

Component	Points	How it is assessed
Part 1 — automated	70	
Builds cleanly	5	make succeeds; -2 if it compiles with warnings
Structural correctness	55	13 weighted cases; within each, node set 30 %, parent/child edges 35 %, exit statuses 20 %, INFO facts 15 %, with structural predicates applied on top
Rejects bad arguments	5	eight malformed invocations must exit non-zero and print no records
Every child reaped	5	no <code>NODE</code> without a matching <code>REAP</code> on the heavy cases
Part 2 — written report	30	
Q1 <code>_exit</code> vs <code>exit</code>	5	the line demonstrably disappears; implication for <code>return</code> stated
Q2 ordering	5	counts from ≥ 10 runs, correct explanation, two ordering mechanisms
Q3 orphans and zombies	10	complete contrast table; three ways to avoid zombies
Video recording	10	high quality video with voice over explanations of each subcommand

7.1 Common ways to lose points

- **The child keeps forking** — no `_exit()` at the end of the child branch. Costs most of `fanout` and `tree`.
- **Never reaping**, so children are orphaned to PID 1 and their `ppid` records come out as 1. Costs the edge checks *and* the leak check.

- **Reaping with ``wait()`` in a loop when order matters** — use `waitpid(pid, ...)` when you need to know *which* child you collected.
- **Forgetting ``WIFSIGNALED``**, so the killed child is reported as `exited:0`.
- **``exec`` in the wrong process**, changing the PID and defeating the whole demonstration.
- **``orphan`` reporting the same ppid twice** — the middle process died before the grandchild ever looked.
- A report with written content but no captured output.

8 Collaboration and AI Policy

This assignment may be completed either **individually or in pairs**. You are encouraged to discuss general concepts, such as what `fork()` returns or what a zombie process is, with other students. However, all submitted work must be your own (or your pair's). You may not view, share, copy, or collaborate on code outside your approved pair.

You may use AI assistants and online sources for **understanding**: explaining a concept, interpreting a compiler error, or reviewing code you have already written. You may **not** submit code you cannot explain line by line. If asked in office hours why your child process calls `_exit` rather than `exit`, you must be able to answer.

Disclosure is required. In [README.md](#), list every source of help — people, tools, and what each contributed. Honest disclosure of substantial AI use costs you no marks. Undisclosed use, or code you cannot explain, is an academic-integrity violation.

END
